

教育本体构建系统：4人完整任务 + 统一接口规范

最终版：任务分配、代码文件、函数签名、输入输出、state约定全部统一

A（后端主控：API + 流程编排 + 系统整合）

角色定位：系统总控、后端接口、LangGraph流程编排。

负责代码文件：

backend/main.py	# FastAPI入口
backend/api/routes.py	# API总路由
backend/api/upload.py	# 上传接口
backend/api/generate.py	# 生成接口
backend/api/export.py	# 导出接口
backend/core/workflow.py	# LangGraph/流程控制核心
backend/config.py	# 配置文件

功能任务：

- FastAPI后端搭建
- 上传/生成/导出接口
- 调用所有模块
- 统一数据流：PDF -> 清洗 -> 匹配 -> LLM -> OWL
- 系统联调
- Git主分支管理

B（AI核心：LLM + 本体生成）

角色定位：大模型调用与 Ontology 生成核心。

负责代码文件：

backend/modules/ontology_builder.py	# 本体生成
backend/services/llm_service.py	# LLM调用
backend/services/prompt.py	# Prompt管理
backend/models/ontology_schema.py	# 本体结构定义

功能任务：

- 调用大模型API
- Prompt设计与优化
- 生成Ontology JSON
- 本体结构设计
- AI输出优化

输出格式：

```
{
  "classes": [],
  "properties": [],
  "relations": []
}
```

C（数据处理：PDF解析 + 清洗）

角色定位：数据入口与结构化处理。

负责代码文件：

backend/modules/pdf_parser.py	# PDF解析
backend/modules/data_cleaner.py	# 数据清洗
backend/utils/file_utils.py	# 文件处理工具

功能任务：

- PDF解析（pdfplumber）

- 表格提取
- 文本提取
- 数据清洗（去噪/格式统一）

输出格式：

```
{
  "raw_text": "...",
  "tables": []
}
```

D（工程整合：匹配 + OWL + 前端）

角色定位：数据工程、本体优化、前端开发。

后端负责代码文件：

```
backend/modules/schema_matcher.py    # 模式匹配
backend/modules/ontology_aligner.py  # 本体对齐
backend/modules/owl_generator.py     # OWL生成
backend/utis/text_utils.py           # 字符串工具
```

- 字段相似度匹配
- 去重
- ontology合并
- OWL文件生成：output.owl

前端部分（Vue3）：

```
frontend/src/views/Upload.vue    # 上传页面
frontend/src/views/Result.vue   # 结果展示
frontend/src/api/request.js      # API请求
frontend/src/App.vue             # 主页面
```

- 上传PDF
- 点击生成本体
- 显示JSON结果
- 下载OWL文件

四人完整任务总表

模块	A	B	C	D
FastAPI接口	✓			
LangGraph流程	✓			
PDF解析			✓	
数据清洗			✓	
LLM调用		✓		
本体生成		✓		
模式匹配				✓
本体对齐				✓
OWL导出				✓
Vue3前端				✓
系统测试	✓	✓	✓	✓

数据流（保证大家不写乱代码）

```
C: PDF -> raw_text
      v
D: schema匹配 + 清洗
      v
B: LLM生成本体
      v
D: OWL导出
      v
A: API统一返回
      v
D: 前端展示
```

这个版本的优点

- 每个人都有代码文件
- 后端不混乱（模块清晰）
- AI模块独立（加分点）
- 前端完整（不会被扣分）
- 可以真实多人协作开发

统一接口规范

全部文件统一接口规范：函数签名、输入输出、state约定；保证4个人不会写冲突，LangGraph / FastAPI / 前端全部能对齐。

一、统一核心规则

1. 统一状态对象（LangGraph State）

```
state = {  
    "file_path": str,  
    "raw_text": str,  
    "clean_data": dict,  
    "ontology": dict,  
    "owl_file": str  
}
```

2. 所有函数必须满足

```
def func_name(state: dict) -> dict:  
    return state
```

二、A模块（API + 流程控制）

backend/api/upload.py

```
POST /upload  
  
def upload_pdf(file) -> dict:  
    """  
    输入: PDF文件  
    输出:  
    {  
        "file_path": str  
    }  
    """
```

backend/api/generate.py

```
POST /generate  
  
def generate_ontology(file_path: str) -> dict:  
    """  
    输入: PDF路径  
    输出:  
    {  
        "ontology": dict,  
        "owl_file": str  
    }  
    """
```

backend/api/export.py

```
GET /export  
  
def export_owl(file_path: str) -> dict:  
    """  
    输出OWL下载路径  
    """
```

backend/core/workflow.py

```
def run_workflow(state: dict) -> dict:  
    """  
    完整流程:  
    PDF -> 清洗 -> 匹配 -> LLM -> OWL  
    """
```

三、C模块（PDF解析）

```
# backend/modules/pdf_parser.py
def extract_pdf(file_path: str) -> str:
    """
    输入: PDF路径
    输出: raw_text
    """

# backend/modules/data_cleaner.py
def clean_data(raw_text: str) -> dict:
    """
    输入: 文本
    输出: 结构化数据
    """
```

四、B模块（LLM + 本体生成）

```
# backend/modules/ontology_builder.py
def build_ontology(clean_data: dict) -> dict:
    """
    输入: 清洗后的数据
    输出: ontology结构
    """

# backend/services/llm_service.py
def call_llm(prompt: str) -> str:
    """
    输入: prompt
    输出: LLM结果（字符串）
    """

# backend/services/prompt.py
def build_prompt(data: dict) -> str:
    """
    输入: 数据
    输出: prompt字符串
    """
```

五、D模块（匹配 + OWL）

```
# backend/modules/schema_matcher.py
def match_schema(data: dict) -> dict:
    """
    输入: 结构化数据
    输出: 统一字段数据
    """

# backend/modules/ontology_aligner.py
def align_ontology(ontology: dict) -> dict:
    """
    输入: ontology
    输出: 优化后的ontology
    """

# backend/modules/owl_generator.py
def generate_owl(ontology: dict) -> str:
    """
    输入: ontology
    输出: owl文件路径
    """
```

六、LangGraph节点统一规范

```
def node_name(state: dict) -> dict:
    return state

extract_node -> extract_pdf()
clean_node -> clean_data()
match_node -> match_schema()
llm_node -> build_ontology()
owl_node -> generate_owl()
```

七、前端统一接口（D模块）

```
// frontend/src/api/request.js
export function uploadPDF(file)

export function generateOntology(filePath)

export function exportOWL()
```

八、完整数据流

```
uploadPDF
  v
  file_path
  v
  extract_pdf()
  v
  clean_data()
  v
  match_schema()
  v
  build_ontology()
  v
  align_ontology()
  v
  generate_owl()
  v
  frontend display
```

九、防冲突规则

- 禁止改别人函数名
- 禁止改输入输出结构
- 禁止随意改state字段
- 只允许在自己模块内部写实现
- 必须严格遵守接口

十、接口设计的好处

- 模块完全解耦
- 可以并行开发
- LangGraph可直接接入
- 前后端统一数据结构
- 工程规范完整